Building Reinforcement Learning Environments

A working manual, from first principles to a tested environment

Opol Charis
Department of Computing and Technology, Uganda Christian University

October 2026

Abstract

A reinforcement learning (RL) agent learns only what its environment lets it experience, so the environment is where most of the design decisions of an RL project are made. This manual explains what an environment is, the mathematics behind it, and the Gymnasium interface that most RL libraries expect. It then builds a small incident-triage environment line by line, explains each design decision in depth (observations, actions and masking, rewards, episode endings, randomness and constraints), connects the environment to the DQN and PPO learners of Stable-Baselines3, and shows how to test an environment before trusting any result trained on it. Every code listing is a file that has been run. The final section maps the manual onto the environment of this project.

Contents

1 What an RL Environment Is **2**

2 The Mathematics Behind an Environment **2**

2.1 Markov decision processes 2

2.2 Return, policy and value 3

2.3 Partial observability 3

3 The Six Design Decisions **4**

4 Gymnasium: The Standard Interface **4**

4.1 Spaces 4

4.2 The two ways an episode ends 5

4.3 Seeding and randomness 5

4.4 Wrappers and registration 5

5 A Worked Example: A Toy Triage Environment **5**

6 Design Decisions in Depth **7**

6.1 Observations 7

6.2 Actions and action masking 7

6.3 Rewards 8

6.4 Episode structure 9

6.5 Randomness and controlled difficulty 9
6.6 Constraints and the safety mechanism 9

7 Connecting a Learner **10**

8 Testing an Environment **11**

9 Common Mistakes **13**

10 From This Manual to Our Environment **13**

1 What an RL Environment Is

In supervised learning, a model learns from a fixed dataset of examples with correct answers. In reinforcement learning there is no dataset. An **agent** learns by acting in an **environment**: it observes the situation, chooses an action, and receives a new observation and a numerical **reward**. Over many such interactions it learns a **policy**, a rule for choosing actions, that collects as much reward as possible over time [1].

The environment is therefore the agent's whole world. It decides what the agent can see, what it can do, what happens when it does it, and what counts as good. Two consequences follow, and they are the reason this manual exists:

1. **The environment defines the problem.** A learner cannot be better than the problem it is given. If the reward pays for the wrong thing, the agent will learn the wrong thing very well.
2. **Environment bugs look like agent behaviour.** If a check silently returns the wrong result, the agent will learn to ignore that check, and the results will report this as something the agent "discovered". Testing the environment comes before training.

Figure 1 shows the interaction loop. One pass through the loop is a **step**. A sequence of steps from a starting state to an end is an **episode**. In this project one episode is the triage of one incident.

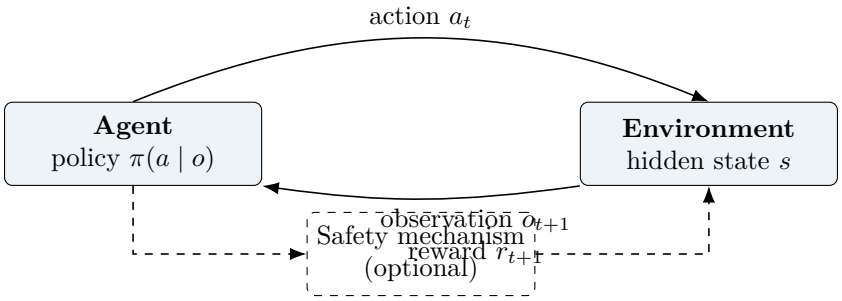


Figure 1: The agent–environment loop. The agent never sees the hidden state, only observations. A safety mechanism, where present, sits between the agent's choice and its execution.

2 The Mathematics Behind an Environment

2.1 Markov decision processes

The standard model of an environment is the **Markov decision process** (MDP), a tuple (S, A, P, R, γ) [1]:

- S, the set of **states**: everything that matters about the situation.
- A, the set of **actions** the agent can take.
- $P(s' \mid s, a)$, the **transition** probabilities: where an action leads.
- $R(s, a, s')$, the **reward**: an immediate number for each transition.
- $\gamma \in [0, 1]$, the **discount**: how much later rewards count relative to immediate ones.

The word *Markov* means that the current state contains everything needed to predict what happens next; the history adds nothing. This property matters in practice: if the observation you give the agent is not Markov, a feed-forward network cannot learn a good policy from it.

2.2 Return, policy and value

The agent tries to maximise the **return**, the discounted sum of future rewards from step t:

$$G_t = r_{t+1} + \gamma\, r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{T-t-1} \gamma^k\, r_{t+k+1}. \tag{1}$$

A **policy** $\pi(a \mid s)$ gives the probability of each action in each state. The **action value** $Q^\pi(s, a)$ is the expected return from taking action a in state s and following π afterwards. The optimal action values satisfy the **Bellman optimality equation**:

$$Q^*(s, a) = \mathbb{E}\big[\, r + \gamma \max_{a'} Q^*(s', a') \;\big|\; s, a \big]. \tag{2}$$

Value-based methods such as Q-learning and the deep Q-network (DQN) [9] learn Q by repeatedly nudging their estimate towards the right-hand side of Equation (2). Once Q is known, the policy is simply "take the action with the highest Q". Notice the $\max_{a'}$: it is the reason invalid actions must also be excluded from the learning target, not only from action selection (Section 6.2).

2.3 Partial observability

In many real problems the agent cannot see the state. A doctor does not see the disease, only symptoms and test results; an IT technician does not see the cause of an incident, only error messages and check results. This is a **partially observable** MDP (POMDP) [2]: the agent receives an **observation** o drawn from $O(o \mid s)$ instead of s itself.

In a POMDP the agent should act on its **belief**, the probability distribution over hidden states given everything observed so far. With a hidden cause c and observed evidence $e_1, \ldots, e_n$ that are independent given the cause, Bayes' rule gives

$$b(c) = P(c \mid e_1, \ldots, e_n) \;\propto\; P(c) \prod_{i=1}^{n} P(e_i \mid c). \tag{3}$$

There are two ways to build an agent for a POMDP. One is to compute the belief explicitly and act on it; this is what the threshold baseline of this project does. The other is to give the agent an observation that *summarises the evidence* and let a neural network learn to act on it; this is what the DQN agent does. The second works only if the summary is sufficient, which is why the design of the observation (Section 6.1) matters so much.

Key point. An environment is an MDP or POMDP written as code. Every choice you make in code (what goes in the observation, what the actions are, what the reward pays for, when an episode ends) is a choice of S, A, P, R and γ.

3 The Six Design Decisions

Every environment answers six questions. Table 1 lists them with the answers this project gives. The rest of the manual explains how to answer each one well.

Table 1: The six design decisions of an environment.

Decision	Question	This project
Hidden state	What is true but unseen?	The cause of the incident; all check results
Observation	What can the agent see, and how is it encoded?	Status of each check, symptoms, context, step count, blocked flag
Actions	What can the agent do?	Run one of 20 checks, recommend one of about 15 fixes, escalate
Transitions	What happens after each action?	A check reveals its result; a fix or escalation ends the episode
Reward	What counts as good?	+ correct fix; − wrong fix; − check cost; − escalation
Episode end	When does it stop?	Fix, escalation, forced escalation, or the step limit

4 Gymnasium: The Standard Interface

Gymnasium is the maintained successor to OpenAI Gym and the standard interface for RL environments in Python [3]. If an environment follows it, any compatible library (Stable-Baselines3, sb3-contrib, RLlib, CleanRL) can train on it without changes. An environment is a class that inherits from `gymnasium.Env` and provides the members in Table 2.

Table 2: The Gymnasium environment interface (version 1.x).

Member	Purpose
`action_space`	The set of valid actions, as a `Space` object. Set in `__init__`.
`observation_space`	The shape, type and bounds of observations. Set in `__init__`.
`reset(seed, options)`	Starts a new episode. Returns `(observation, info)`. Must call `super().reset(seed=seed)` first, which seeds `self.np_random`.
`step(action)`	Applies one action. Returns `(observation, reward, terminated, truncated, info)`.
`render()`	Optional. Shows the current state, for debugging or demonstration.
`close()`	Optional. Releases resources such as windows or files.
`metadata`	Class attribute, e.g. the supported `render_modes`.

4.1 Spaces

Spaces describe what valid actions and observations look like. The common ones are:

- `Discrete(n)`: integers $0, \ldots, n-1$. The usual action space for choosing one of n options. DQN supports only this kind of action space.
- `Box(low, high, shape, dtype)`: real-valued arrays with bounds. The usual observation space. A flat `Box` works with the standard `"MlpPolicy"`.
- `MultiBinary(n)`, `MultiDiscrete([n1, n2, ...])`: vectors of binary or integer values.
- `Dict` and `Tuple`: combinations of other spaces. They need `"MultiInputPolicy"` in Stable-Baselines3; a single flat `Box` is simpler.

Every observation returned by `reset` and `step` must lie in `observation_space`, with the same `dtype` (usually `np.float32`). A mismatch is the most common error that the environment checker reports.

4.2 The two ways an episode ends

Gymnasium distinguishes two endings, and the difference changes what the agent learns [5]:

- `terminated=True`: the episode reached a *true end of the task*, such as a fix being applied. There is no future, so the learning target is just the final reward.
- `truncated=True`: the episode was *cut off for reasons outside the task*, such as a time limit imposed only to keep training runs short. The task would have continued, so the learner should still bootstrap from the value of the next state.

If a time limit is part of the problem, and the agent can see how much time remains, then reaching it is a termination. In this project the step limit is part of the task (reaching it means the incident is escalated, at a cost) and the step count is in the observation, so it is a termination. Treating it as truncation would teach the agent that stalling is free.

4.3 Seeding and randomness

All randomness inside an environment must come from `self.np_random`, the generator that `super().reset(seed=seed)` creates. Never use Python's `random` module or `np.random` directly inside an environment. With this rule, `reset(seed=7)` always produces the same incident, which makes bugs reproducible and allows every policy to be evaluated on exactly the same incidents.

4.4 Wrappers and registration

A **wrapper** is an object that wraps an environment and changes one aspect of it without editing it: `ObservationWrapper` changes observations, `RewardWrapper` changes rewards, `ActionWrapper` changes actions. Stable-Baselines3's `Monitor` wrapper records episode returns and lengths. Wrappers are how you add things that differ between experiments (for example, an extra penalty for the penalty-only baseline) without making copies of the environment. **Registration** lets the environment be created by name. It is optional but convenient:

```
gymnasium.register(id="Triage-v0", entry_point="triage.env:TriageEnv")
env = gymnasium.make("Triage-v0")
```

5 A Worked Example: A Toy Triage Environment

The best way to understand an environment is to build a small one completely. Listing 1 is a toy version of this project's environment: three hidden causes, two checks with different costs, three fixes and one escalate action. The likelihood matrix `theta` says how often each check is positive under each cause. Check 0 is good at recognising cause 0, check 1 is good at recognising cause 1, and cause 2 is ambiguous on both: this is deliberate, so that the agent has a reason to escalate.

Listing 1: A complete toy triage environment (`code/toy_triage_env.py`).

```
import numpy as np
import gymnasium as gym
from gymnasium import spaces


class ToyTriageEnv(gym.Env):
```

```
    """Three hidden causes, two diagnostic checks, one escalate action."""

    metadata = {"render_modes": ["ansi"]}

    NOT_REQUESTED, NEGATIVE, POSITIVE = 0, 1, 2

    def __init__(self, r_correct=10.0, cost_wrong=10.0, cost_esc=3.0,
                 check_costs=(0.2, 0.5), max_steps=4, render_mode=None):
        # theta[i, j] = P(check i is positive | cause j)
        self.theta = np.array([[0.9, 0.1, 0.5],
                               [0.2, 0.8, 0.6]])
        self.prior = np.array([0.5, 0.3, 0.2])          # P(cause j)
        self.n_checks, self.n_causes = self.theta.shape
        self.r_correct, self.cost_wrong, self.cost_esc = r_correct, cost_wrong, cost_esc
        self.check_costs = np.asarray(check_costs)
        self.max_steps = max_steps
        self.render_mode = render_mode

        # Actions: [0, n_checks) run a check; [n_checks, n_checks + n_causes) recommend
        # the fix for a cause; the last action escalates.
        self.n_actions = self.n_checks + self.n_causes + 1
        self.ESCALATE = self.n_actions - 1
        self.action_space = spaces.Discrete(self.n_actions)
        # Observation: one-hot status of each check (3 values) + normalised step count.
        self.observation_space = spaces.Box(0.0, 1.0, shape=(3 * self.n_checks + 1,),
                                            dtype=np.float32)

    # ----- helpers ---------------------------------------------------------
    def _obs(self):
        obs = np.zeros(self.observation_space.shape, dtype=np.float32)
        for i, s in enumerate(self.status):
            obs[3 * i + s] = 1.0
        obs[-1] = self.t / self.max_steps
        return obs

    def action_masks(self):
        mask = np.ones(self.n_actions, dtype=bool)
        mask[:self.n_checks] = self.status == self.NOT_REQUESTED   # no repeated checks
        return mask

    # ----- Gymnasium API ---------------------------------------------------
    def reset(self, *, seed=None, options=None):
        super().reset(seed=seed)                            # seeds self.np_random
        self.cause = int(self.np_random.choice(self.n_causes, p=self.prior))
        # Draw every check result once, so a repeated check gives the same answer.
        self.results = self.np_random.random(self.n_checks) < self.theta[:, self.cause]
        self.status = np.full(self.n_checks, self.NOT_REQUESTED)
        self.t = 0
        return self._obs(), {"cause": self.cause}

    def step(self, action):
        action = int(action)
        self.t += 1
        terminated, info = False, {}

        if action < self.n_checks:                          # run a check
            self.status[action] = self.POSITIVE if self.results[action] else self.NEGATIVE
            reward = -float(self.check_costs[action])
        elif action < self.ESCALATE:                        # recommend a fix
            correct = (action - self.n_checks) == self.cause
            reward = self.r_correct if correct else -self.cost_wrong
            terminated, info["outcome"] = True, "correct" if correct else "wrong"
        else:                                               # escalate
            reward = -self.cost_esc
            terminated, info["outcome"] = True, "escalated"
```

```
72
73          if not terminated and self.t >= self.max_steps:    # step limit = forced escalation
74              reward -= self.cost_esc
75              terminated, info["outcome"] = True, "step_limit"
76
77          return self._obs(), reward, terminated, False, info
78
79      def render(self):
80          names = {0: "?", 1: "-", 2: "+"}
81          return f"t={self.t} checks=[{' '.join(names[s] for s in self.status)}]"
```

Read the listing with Section 6 beside it. Four details deserve attention now:

- **Lines 35–40** build the observation as a one-hot vector: for each check, exactly one of *not requested*, *negative*, *positive* is 1.
- **Line 52** draws every check result once at the start of the episode. Running a check twice therefore gives the same answer, as it would in reality, and the observation stays Markov.
- **Lines 42–45** define `action_masks()`: checks already run are unavailable. Learners that support masking will never choose them.
- **Lines 73–75** implement the step limit as a forced escalation that costs the agent the escalation penalty, and they return `terminated=True` for the reason given in Section 4.2.

6 Design Decisions in Depth

6.1 Observations

The observation must contain everything the agent needs to act well, encoded so that a network can use it.

Sufficiency. Ask: could a perfect decision-maker act optimally from this observation alone? In triage, the results of the checks so far, the symptoms and the context are sufficient, because check results do not change during an episode. The step count is also needed, because the value of running another check depends on how many steps remain.

Encoding. Use **one-hot encoding** for categories. Encoding a check's status as $0, 1, 2$ in a single number would tell the network that "positive" is twice "negative", which is meaningless; three separate 0/1 entries carry no such false order. Scale numbers to roughly $[0, 1]$ (divide the step count by the step limit), because networks learn poorly from inputs of very different sizes.

What not to include. Never put the hidden state (the true cause) or anything computed from it into the observation. This is **information leakage**, and it produces agents that look perfect and are worthless. The toy environment returns the cause in the `info` dictionary for testing and logging only; learners do not read `info`.

6.2 Actions and action masking

A `Discrete` action space with a fixed numbering is simplest: in the toy, actions 0–1 are checks, 2–4 are fixes, 5 is escalate. In this project, with 20 checks and 15 causes, actions 0–19 are checks, 20–34 are fixes and 35 is escalate: $20 + 15 + 1 = 36$.

Some actions are **invalid** in some states, such as repeating a check. There are three ways to handle this [6]:

1. **Penalise** invalid actions with a negative reward. Simple, but the agent wastes training time learning what is invalid, and it may still choose invalid actions.
2. **Ignore** them: treat an invalid action as a wasted step. Safe, but just as slow to learn.

3. **Mask** them: the learner is told which actions are valid and never chooses the others. This is the most efficient, and Huang and Ontañón show that it scales to large action spaces where penalties fail [6].

Masking must be applied in two places for value-based learners: in *choosing* the action, and in the $\max_{a'}$ of the learning target (Equation 2). Otherwise the agent learns values that assume it can take actions it cannot. Section 7 shows both.

Key point. Do not mask actions that the *rules* forbid before the agent proposes them. If rule-forbidden fixes were masked in advance, the agent could never propose a violation, and the study could not measure whether it learns the rules. In this project only repeated checks are masked in advance; rule violations are caught by the safety mechanism *after* the proposal, and the blocked action is masked only for the agent's second try.

The environment must still behave sensibly if an invalid action arrives, because some code paths (random exploration in libraries without masking, your own tests) will send one. In the toy, a repeated check simply returns the same result and costs the check again.

6.3 Rewards

The reward is the only way to tell the agent what you want. Three principles apply.

Reward outcomes, not methods. Pay for a correct fix and charge for costs; do not pay the agent for "running the right check", because you would be imposing your own strategy and the agent might learn to collect check rewards instead of resolving incidents. This failure, where an agent maximises the reward without doing the task, is called **reward hacking**. If extra guidance is needed, **potential-based shaping**, a reward of the form $\gamma\,\Phi(s') - \Phi(s)$ for some function Φ of the state, is the one form proven not to change which policy is optimal [7].

The reward values encode a decision threshold. Suppose the agent stops investigating and its belief that the most likely cause is correct is p. Recommending that fix has expected reward $p\,r_{\text{correct}} - (1-p)\,\lambda_{\text{wrong}}$; escalating has reward $-\lambda_{\text{esc}}$. Recommending is better exactly when

$$p \;>\; p^* = \frac{\lambda_{\text{wrong}} - \lambda_{\text{esc}}}{r_{\text{correct}} + \lambda_{\text{wrong}}}. \tag{4}$$

With the starting values $r_{\text{correct}} = 10$, $\lambda_{\text{wrong}} = 10$ and $\lambda_{\text{esc}} = 3$, the threshold is $p^* = 7/20 = 0.35$: the optimal agent will recommend a fix it believes is only 35% likely to be right. That is probably too bold for IT operations. Raising the cost of a wrong fix to 20 gives $p^* = 17/30 \approx 0.57$; raising it to 40 gives $p^* = 37/50 = 0.74$. Equation (4) turns the abstract question "what should the rewards be?" into a concrete one that the Directorate can answer: *how sure should someone be before applying a fix rather than escalating?*

The toy environment makes the effect visible. Because it is tiny, its exact optimal policy can be computed by searching every possible sequence of check results (`code/optimal_toy.py`). Table 3 shows that optimal policy for three costs of a wrong fix, all else equal. With $\lambda_{\text{wrong}} = 10$ the best possible agent *never* escalates and is wrong 28% of the time; it is not misbehaving, it is doing exactly what the rewards ask. As a wrong fix becomes more costly, the optimal agent escalates more and errs less.

Table 3: Optimal behaviour of the toy environment as the cost of a wrong fix rises ($r_{\text{correct}} = 10$, $\lambda_{\text{esc}} = 3$; outcome shares over 1,000 incidents).

λ_{wrong}	p^*	Optimal return	Correct	Wrong	Escalated
10	0.35	4.20	71.9%	28.1%	0.0%
20	0.57	1.85	67.2%	20.3%	12.5%
40	0.74	-0.50	57.6%	10.9%	31.5%

Check costs set how much evidence is worth buying. A check is worth running only if the expected improvement in the final decision exceeds its cost. If checks are free, the agent will run all of them; if they are expensive relative to the threshold, it will run none. Pilot runs should confirm that the agent runs some checks, and fewer when the evidence is already clear.

Scale and discount. Keep rewards roughly in $[-50, 50]$; very large values destabilise learning. Episodes here are short and the check costs already penalise delay, so $\gamma = 0.99$ (the library default) or $\gamma = 1$ both work. Report the choice.

6.4 Episode structure

Every episode must end. Give each one an explicit step limit and test that it is reached (Listing 4). Decide for each ending whether it is a termination or a truncation (Section 4.2), and record the reason in `info` (`"outcome"` in the toy) so that the evaluation can count correct fixes, wrong fixes, escalations and step-limit escalations separately.

6.5 Randomness and controlled difficulty

A simulator is valuable because its difficulty can be controlled. In the toy, randomness enters at `reset`: the cause is drawn from the prior and the check results from `theta`. Difficulty controls are parameters of the constructor that change those draws. This project has three:

- **Missing evidence** (p_{miss}): at `reset`, mark each check as unavailable with this probability. Running it returns *unavailable*, a fourth status in the one-hot encoding.
- **Unreliable evidence** (p_{noise}): at `reset`, flip each drawn result with this probability.
- **Overlapping evidence** (ω): in `__init__`, blend each cause's column of `theta` towards its category average, $\theta'_{ij} = (1 - \omega)\,\theta_{ij} + \omega\,\bar{\theta}_i$.

Passing these through `reset(options={...})` instead allows a different difficulty in every episode, which is how the randomised training regime is implemented.

6.6 Constraints and the safety mechanism

A **shield** is a component that checks each action against rules and prevents violations [8]. There are two places to put it:

- **As a wrapper** around the environment. Clean when the shield only replaces actions, but awkward when the agent must *see* that it was blocked.
- **Inside `step`**, behind a constructor switch. Simpler when the shield changes the observation (the *blocked* flag) and the mask.

This project uses the second. In `step`, before executing an action: (1) evaluate the rules; (2) if the action is permitted, execute it; (3) if it is forbidden and this is the first block, do not execute it, set the blocked flag, mask that action, give zero reward and return; (4) if it is a second consecutive block, end the episode as a *forced escalation*. The rules themselves should be data (a list loaded from a configuration file) evaluated by a small function such as `is_forbidden(action, context, evidence)`, so that they can be read and reviewed without reading the agent's code. A switch `shield=False` produces the unconstrained baseline, and a wrapper that subtracts a penalty for each violation produces the penalty-only baseline, from the same environment class.

7 Connecting a Learner

Stable-Baselines3 (SB3) provides tested implementations of the standard algorithms [4], and **sb3-contrib** adds `MaskablePPO`, a PPO that accepts action masks. SB3's `DQN` does not support masks, so Listing 2 extends it. The environment appends its mask to the end of the observation (a small `ObservationWrapper` does this). The Q-network then sets the value of every masked action to a very large negative number. Because the target network also reads the mask from the *next* observation, this one change covers both action selection and the max in the learning target. Two further overrides make random exploration respect the mask.

Listing 2: DQN with action masking (`code/masked_dqn.py`).

```python
"""DQN with action masking for Stable-Baselines3.

The environment appends its action mask (1 = allowed) to the end of the observation.
Masking the Q-values inside the network covers both action selection and the max in
the learning target, because the target network sees the mask in the next observation.
"""
import numpy as np
from stable_baselines3 import DQN
from stable_baselines3.dqn.policies import DQNPolicy, QNetwork


class MaskedQNetwork(QNetwork):
    def forward(self, obs):
        q = super().forward(obs)
        mask = obs[:, -int(self.action_space.n):]
        return q.masked_fill(mask < 0.5, -1e9)


class MaskedDQNPolicy(DQNPolicy):
    def make_q_net(self):
        net_args = self._update_features_extractor(self.net_args, features_extractor=None)
        return MaskedQNetwork(**net_args).to(self.device)


class MaskedDQN(DQN):
    def _masked_random(self, obs):
        n = int(self.action_space.n)
        masks = np.asarray(obs).reshape(-1, np.asarray(obs).shape[-1])[:, -n:].astype(np.int8)
        return np.array([self.action_space.sample(mask=m) for m in masks])

    def predict(self, observation, state=None, episode_start=None, deterministic=False):
        if not deterministic and np.random.rand() < self.exploration_rate:
            return self._masked_random(observation), state       # masked epsilon-greedy
        return self.policy.predict(observation, state, episode_start, deterministic)

    def _sample_action(self, learning_starts, action_noise=None, n_envs=1):
        if self.num_timesteps < learning_starts:                  # masked warm-up
            action = self._masked_random(self._last_obs)
            return action, action
        return super()._sample_action(learning_starts, action_noise, n_envs)
```

Training and evaluation then take a few lines (Listing 3); each run takes one to three minutes on a laptop CPU. Note `net_arch=[128, 128]`, the two hidden layers of 128 units chosen for this project; the evaluation seeds start at 10,000 so that the agent is evaluated on incidents it never trained on.

Listing 3: Training both learners on the toy environment (excerpt of `code/train_toy.py`).

```python
if __name__ == "__main__":
    # DQN: mask travels inside the observation.
    dqn_env = Monitor(MaskInObservation(ToyTriageEnv()))
    dqn = MaskedDQN(MaskedDQNPolicy, dqn_env, policy_kwargs=dict(net_arch=[128, 128]),
                    learning_starts=1_000, exploration_fraction=0.3, seed=0, verbose=0)
```

```
    dqn.learn(total_timesteps=100_000)
    print("DQN:", evaluate(dqn, MaskInObservation(ToyTriageEnv())))

    # PPO: mask supplied by the ActionMasker wrapper.
    ppo_env = ActionMasker(Monitor(ToyTriageEnv()), lambda e: e.unwrapped.action_masks())
    ppo = MaskablePPO("MlpPolicy", ppo_env, seed=0, verbose=0)
    ppo.learn(total_timesteps=30_000)
    print("PPO:", evaluate(ppo, ActionMasker(ToyTriageEnv(), lambda e: e.unwrapped.action_masks()
    )))
```

Monitoring training. Wrap the training environment in `Monitor` and watch the mean episode return. It should rise and then level off. If it never rises, suspect the environment or the reward before the hyperparameters. SB3's `EvalCallback` evaluates the agent on a separate environment every few thousand steps and saves the best model.

8 Testing an Environment

Test the environment before training any agent on it. Listing 4 shows the five tests used for the toy; each catches a different class of bug.

Listing 4: Tests for the toy environment (`code/test_toy_env.py`).

```python
import numpy as np
from gymnasium.utils.env_checker import check_env
from toy_triage_env import ToyTriageEnv


def test_api():
    check_env(ToyTriageEnv())                  # spaces, dtypes, reset/step signatures, seeding


def test_same_seed_same_episode():
    a, b = ToyTriageEnv(), ToyTriageEnv()
    assert np.array_equal(a.reset(seed=7)[0], b.reset(seed=7)[0])
    assert a.cause == b.cause and np.array_equal(a.results, b.results)


def test_likelihoods_match_theta():
    env, n = ToyTriageEnv(), 20_000
    hits = np.zeros_like(env.theta)
    counts = np.zeros(env.n_causes)
    for k in range(n):
        env.reset(seed=k)
        hits[:, env.cause] += env.results
        counts[env.cause] += 1
    assert np.allclose(hits / counts, env.theta, atol=0.03)
    assert np.allclose(counts / n, env.prior, atol=0.02)


def test_reward_ordering():
    env = ToyTriageEnv()
    assert env.r_correct > -env.cost_esc > -env.cost_wrong


def test_episode_always_ends():
    env = ToyTriageEnv()
    env.reset(seed=0)
    for _ in range(env.max_steps):
        _, _, terminated, truncated, _ = env.step(0)   # keep running the same check
        if terminated or truncated:
            return
    raise AssertionError("episode did not end at the step limit")
```

1. **API conformance.** Gymnasium's `check_env` confirms that spaces, data types, return values and seeding follow the interface.
2. **Determinism.** The same seed must give the same incident. Without this no result can be reproduced.
3. **Statistical correctness.** Over many episodes, the frequency of each cause must match the prior and the frequency of positive checks must match `theta`. This catches indexing errors (rows and columns swapped), which are otherwise invisible.
4. **Reward ordering.** The configured rewards must satisfy $r_{\text{correct}} > -\lambda_{\text{esc}} > -\lambda_{\text{wrong}}$.
5. **Termination.** Every episode must end, even for an agent that repeats one action forever.

Then run **reference policies** and check their scores make sense before training. A *random* policy gives a floor. An *always-escalate* policy gives exactly $-\lambda_{\text{esc}}$ per episode. An *oracle* that is told the cause gives r_{correct}; no agent that cannot see the cause can reach it, so an agent that comes close has almost certainly been given leaked information. In a small environment the *exact optimum* can be computed and is the true ceiling. In the project environment, with 4^{20} possible check histories, it cannot, and the *threshold* policy, which computes the belief of Equation (3) and acts when it is confident, serves as a strong reference instead. It is a heuristic, not a ceiling: a learned agent can beat it.

Results on the toy. Table 4 shows every policy on the same 1,000 unseen incidents. Both learners reach the exact optimum, which confirms that the environment, the masking and the training code work together. Two lessons carry over to the project. First, the threshold heuristic falls short of the optimum because its fixed confidence rule (act above 0.8) is not what the rewards ask for, which is why it is a reference and not a ceiling. Second, the masked DQN needed about 100,000 steps; at 30,000 it still escalated 39% of incidents and scored 1.77, while PPO converged in 30,000. Training length must be set from learning curves, not guessed. These are checks of the toy, not results of this project.

Table 4: All policies on the toy environment, 1,000 unseen incidents (`code/make_toy_results.py`).

Policy	Mean return	Correct	Wrong	Escalated	Step limit
Random (valid actions)	-3.32	25.1%	49.2%	25.7%	0.0%
Always escalate	-3.00	0.0%	0.0%	100.0%	0.0%
Threshold (knows θ)	3.68	71.9%	28.1%	0.0%	0.0%
Masked DQN (100k steps)	4.18	71.9%	28.1%	0.0%	0.0%
MaskablePPO (30k steps)	4.18	71.9%	28.1%	0.0%	0.0%
Exact optimum	4.18	71.9%	28.1%	0.0%	0.0%
Oracle (told the cause)	10.00	100.0%	0.0%	0.0%	0.0%

9 Common Mistakes

Mistake	Consequence and remedy
Hidden state in the observation	Perfect but meaningless agent. Keep the cause in `info` only, and test that the observation does not change when only the cause changes.
Using `np.random` in the environment	Seeds have no effect; nothing is reproducible. Use `self.np_random`.
Wrong `dtype` or bounds	`check_env` fails or learning is silently wrong. Return `np.float32` within the declared bounds.
Time limit as truncation when it is part of the task	Agent learns that stalling is free. Use `terminated` and charge the cost.
Masking only action selection in DQN	Learning targets use impossible actions; values are overestimated. Mask inside the Q-network.
Masking rule-forbidden actions in advance	No violations can occur, so compliance cannot be measured. Mask only after a block.
Unchecked reward values	Agent is too bold or too timid. Compute the threshold of Equation (4) first.
Training and evaluating on the same seeds	Overstated results. Keep separate seed ranges.
Tuning the simulator after seeing the agent's results	The environment is fitted to the agent. Freeze the scenario model before training.

10 From This Manual to Our Environment

The project's environment is the toy of Section 5 grown along the lines of Section 6. Table 5 lists the differences.

Table 5: From the toy environment to the project environment.

Element	Toy	Project
Causes, checks	3 causes, 2 checks, hard-coded	About 15 causes, 20 checks, loaded from the elicitation workbook
Check status	3 values	4 values (adds *unavailable*)
Observation	Checks, step count	Adds symptoms (3 values each), context attributes, blocked flag, action mask
Actions	6	36
Ambiguity	None	p_{miss}, p_{noise}, ω, fixed or drawn per episode
Rules	None	10 rules from a configuration file, checked inside `step`
Reference labels	None	"Should escalate" label in `info`, for evaluation only

Package layout. The code will be organised as follows, so that the scenario and rules are data and every experiment can be traced to its configuration:

```
triage/
  config/   scenario.yaml  rules.yaml  experiments/*.yaml
  scenario.py    # reads the workbook export -> prior, theta, symptom likelihoods, costs
  rules.py       # Rule objects and is_forbidden(action, context, evidence)
  env.py         # TriageEnv(gymnasium.Env): reset, step, action_masks, shield switch
  wrappers.py    # MaskInObservation, PenaltyOnly
  baselines.py   # initial-symptom, scripted, always-escalate, threshold
  masked_dqn.py  # Listing 2
```

```
  train.py  evaluate.py
tests/      test_env.py  test_rules.py  test_baselines.py
```

Build order. Each step is tested before the next begins:

1. **Provisional scenario.** Write `scenario.yaml` by hand with invented values in the four categories, so that development does not wait for the elicitation. Replace it after the session.
2. **Core environment.** Hidden cause, checks, symptoms, context, rewards and the step limit; pass the five tests of Listing 4.
3. **Ambiguity controls.** Add p_{miss}, p_{noise} and ω; test that each changes the statistics as intended.
4. **Reference policies.** Random, always-escalate, oracle and threshold; check their scores are ordered as in Section 8.
5. **Rules and shield.** Implement `rules.py` and the review-then-escalate logic; test each rule with a hand-built incident.
6. **Learners.** Train the masked DQN on the provisional scenario as a smoke test; only then on the frozen elicited scenario.

References

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[2] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial Intelligence*, vol. 101, no. 1–2, pp. 99–134, 1998.

[3] M. Towers *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," arXiv:2407.17032, 2024.

[4] A. Raffin *et al.*, "Stable-Baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

[5] F. Pardo, A. Tavakoli, V. Levdik, and P. Kormushev, "Time limits in reinforcement learning," in *Proc. 35th Int. Conf. Machine Learning (ICML)*, 2018, pp. 4045–4054.

[6] S. Huang and S. Ontañón, "A closer look at invalid action masking in policy gradient algorithms," in *Proc. 35th Int. FLAIRS Conf.*, 2022.

[7] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proc. 16th Int. Conf. Machine Learning (ICML)*, 1999, pp. 278–287.

[8] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 32, no. 1, 2018.

[9] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.